

Segment Tree

Introduction

A **Segment Tree** is a binary tree used to efficiently answer **range queries** (like sum, maximum, minimum) and **point updates** on an array.

- Each node represents a **range [l, r]**
- Leaf nodes represent **single elements**
- Internal nodes store a **function** (sum / max / min) of their children
- **Build:** $O(n)$
- **Query:** $O(\log n)$
- **Update:** $O(\log n)$

Operations :

1. Segment Tree Construction

The segment tree is built by recursively dividing the given array into two halves until each segment contains only one element. Each leaf node of the tree stores the corresponding array element as its sum, minimum, and maximum value. Internal nodes combine information from their left and right child nodes: the sum node stores the addition of child sums, the minimum node stores the smaller value, and the maximum node stores the larger value. This divide-and-conquer approach ensures that every range of the array is represented efficiently, allowing fast query operations later.

Algorithm: BUILD-SEGMENT-TREE

BUILD(node, start, end):

if start == end:

treeSum[node] = A[start]

treeMin[node] = A[start]

treeMax[node] = A[start]

else:

mid = (start + end) // 2

BUILD(2*node, start, mid)

BUILD(2*node+1, mid+1, end)

treeSum[node] = treeSum[2*node] + treeSum[2*node+1]

treeMin[node] = min(treeMin[2*node], treeMin[2*node+1])

treeMax[node] = max(treeMax[2*node], treeMax[2*node+1])

Time Complexity: $O(n)$

Space Complexity: $O(4n)$

2. Range Sum Query

To find the sum of elements in a given range, the segment tree is traversed from the root. If the current segment lies completely outside the query range, it contributes zero to the result. If it lies completely inside the range, the precomputed sum stored at that node is returned directly. If the segment partially overlaps the query range, the query is recursively performed on both left and right child nodes, and their results are added. This avoids visiting unnecessary segments and ensures logarithmic time complexity.

Algorithm: RANGE-SUM

QUERY-SUM(node, start, end, l, r):

if r < start or end < l:

return 0

if l <= start and end <= r:

return treeSum[node]

```

mid = (start + end) // 2
leftSum = QUERY-SUM(2*node, start, mid, l, r)
rightSum = QUERY-SUM(2*node+1, mid+1, end, l, r)

return leftSum + rightSum
Time Complexity: O(log n)

```

3. Range Minimum Query (RMQ)

The range minimum query works by checking how each segment of the tree overlaps with the requested range. If a segment lies fully outside the range, it returns a very large value so it does not affect the result. If it lies completely inside, the stored minimum value of that segment is returned. For partial overlap, the query continues recursively on both child segments, and the minimum of the two returned values is selected. This method efficiently finds the minimum element in a range without scanning all elements.

Algorithm: RANGE-MIN

```

QUERY-MIN(node, start, end, l, r):
    if r < start or end < l:
        return +∞
    if l <= start and end <= r:
        return treeMin[node]

    mid = (start + end) // 2
    leftMin = QUERY-MIN(2*node, start, mid, l, r)
    rightMin = QUERY-MIN(2*node+1, mid+1, end, l, r)

    return min(leftMin, rightMin)
Time Complexity: O(log n)

```

4. Range Maximum Query

The range maximum query follows the same traversal logic as the minimum query but focuses on the largest value. Segments completely outside the query range return a very small value, while segments fully inside return their stored maximum. For partially overlapping segments, the query moves down to both children and computes the maximum of their results. By utilizing stored segment information, the maximum value in any range is obtained in logarithmic time.

Algorithm: RANGE-MAX

```

QUERY-MAX(node, start, end, l, r):
    if r < start or end < l:
        return -∞
    if l <= start and end <= r:
        return treeMax[node]

    mid = (start + end) // 2
    leftMax = QUERY-MAX(2*node, start, mid, l, r)
    rightMax = QUERY-MAX(2*node+1, mid+1, end, l, r)

    return max(leftMax, rightMax)
Time Complexity: O(log n)

```

5. Point Update (Update Index Value)

In a point update operation, a single array element is modified, and the change must be reflected in the segment tree. The update process starts from the root and recursively moves down to the leaf node corresponding to the updated index. Once the leaf value is changed, the update propagates upward, recalculating sum, minimum, and maximum values at each affected internal node. Since only one path from root to leaf is updated, the operation runs efficiently in logarithmic time.

Algorithm: UPDATE

UPDATE(node, start, end, idx, value):

if start == end:

 A[idx] = value

 treeSum[node] = value

 treeMin[node] = value

 treeMax[node] = value

else:

 mid = (start + end) // 2

 if idx <= mid:

 UPDATE(2*node, start, mid, idx, value)

 else:

 UPDATE(2*node+1, mid+1, end, idx, value)

 treeSum[node] = treeSum[2*node] + treeSum[2*node+1]

 treeMin[node] = min(treeMin[2*node], treeMin[2*node+1])

 treeMax[node] = max(treeMax[2*node], treeMax[2*node+1])

6 Update Range in Segment Tree

A **range update in a segment tree** refers to the operation of **modifying all elements within a given index range** $[l, r]$ instead of updating a single position. In this operation, a value is added, set, or modified for every element that lies in the specified range, and the segment tree is updated so that future queries return correct results. Directly updating each element in the range would be inefficient, so **lazy propagation** is used, where updates are postponed and stored at higher-level nodes and applied only when necessary. This technique ensures that range updates and queries both run in $O(\log n)$ time.

Example: If the array is [2, 4, 6, 8, 10] and we apply a range update to **add 3 to range [1, 3]**, the array becomes [2, 7, 9, 11, 10].

Time Complexity: $O(\log n)$

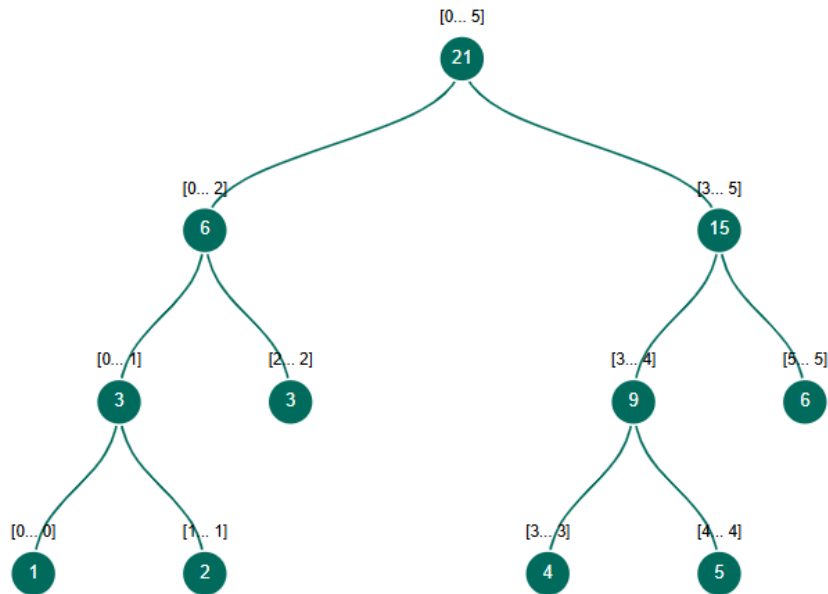
Operation	Time Complexity
Build Segment Tree	$O(n)$
Range Sum Query	$O(\log n)$
Range Min Query	$O(\log n)$
Range Max Query	$O(\log n)$
Point Update	$O(\log n)$
Updaterange	$O(\log n)$

Applications of Segment Tree

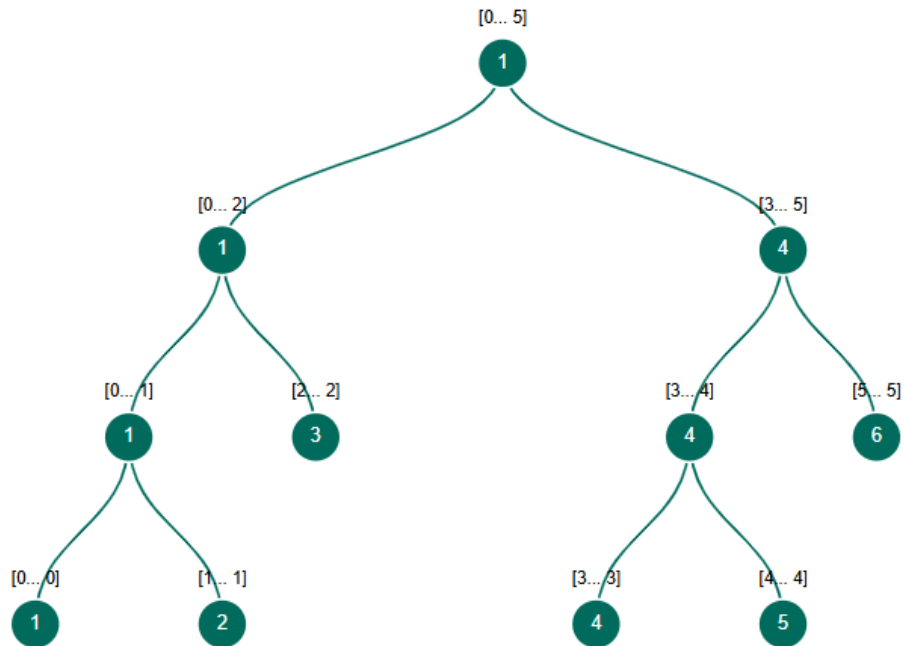
- Range sum / min / max queries
- Competitive programming
- Database query optimization
- Time-series data analysis
- Online query systems

Problem1 :

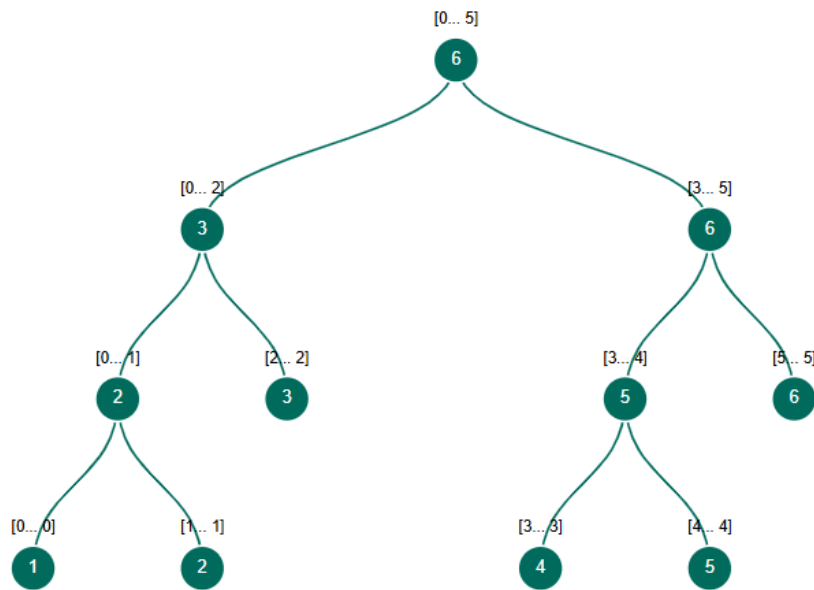
Create Segment Tree using **range sum query** of given elements 1, 2, 3, 4, 5, 6



Create Segment Tree using **range Min query** of given elements 1, 2, 3, 4, 5, 6



Create Segment Tree using **range Max query** of given elements 1, 2, 3, 4, 5, 6



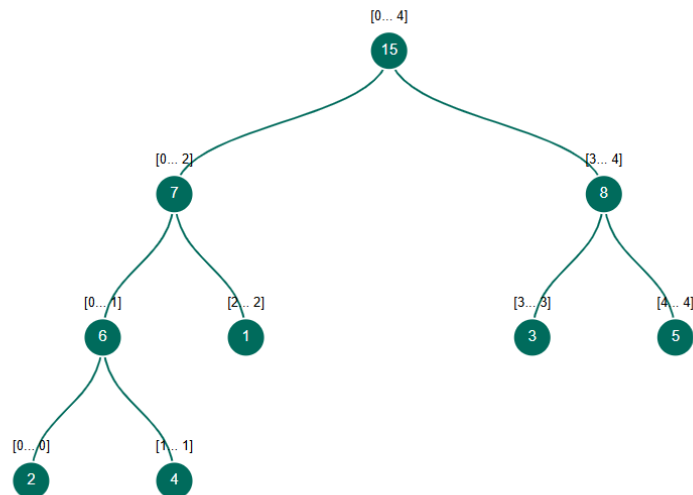
Problem 2:

Create Segment Tree of the following elements 2, 4, 1, 3, 5 using RSQ(RangeSum Query) and update index 2 with 6 value .

Solution:

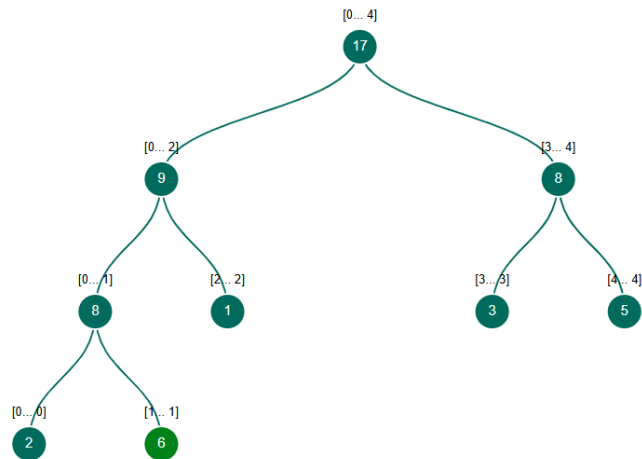
Array elements	2	4	1	3	5
index	0	1	2	3	4

RSQ(RangeSum Query)



update index 2 with 6 value i.e

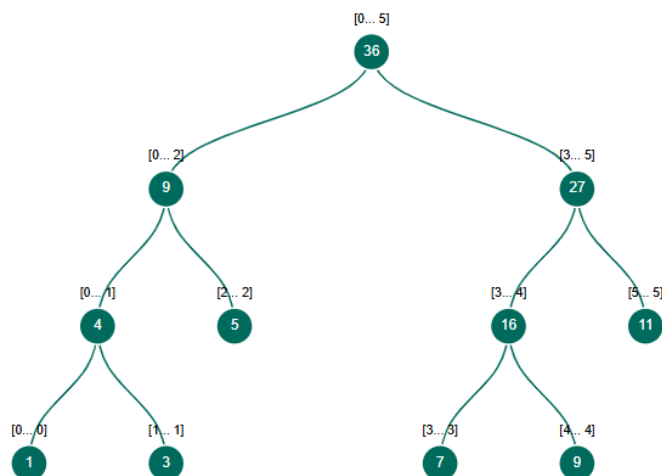
Array elements	2	4	6	3	5
index	0	1	2	3	4



Problem 3: Create Segment Tree of the following elements 1,3,5,7,9,11 using RSQ(RangeSum Query) and update index 2 with 8 value ,update range(1,3)=8 and draw the final segment tree .

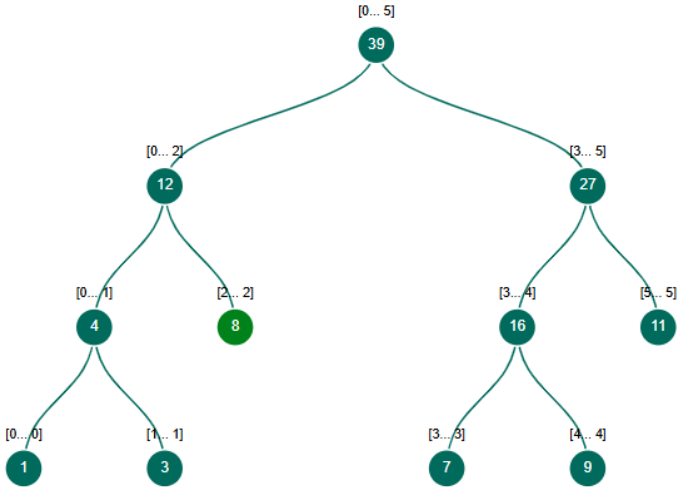
Solution:

Array elements	1	3	5	7	9	11
index	0	1	2	3	4	5



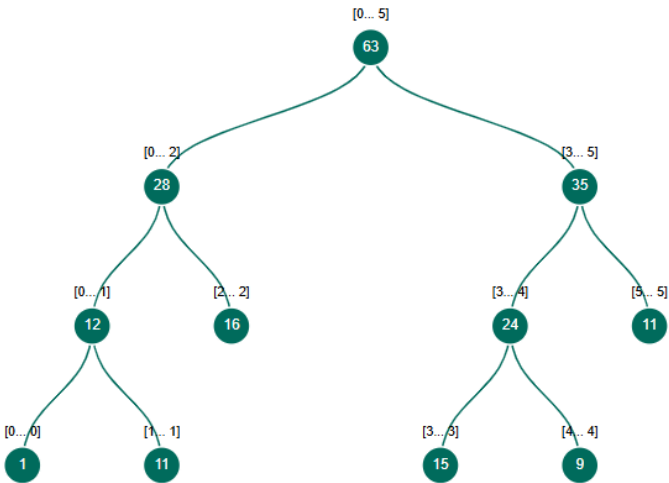
update index 2 with 8 value

Array elements	1	3	8	7	9	11
index	0	1	2	3	4	5



update range(1,3)=8 means

Array elements	1	3	8	7	9	11
index	0	1	2	3	4	5
range(1,3)=8		3+8	8+8	7+8		
New Array elements	1	11	16	15	9	11



Problem 4 : Create a Segment Tree for the following array elements using RSQ (Range Sum Query):

Array: [4, 6, 2, 10, 8, 12]

1. Construct the segment tree using Range Sum Query (RSQ).
2. Perform a point update: update index 2 with value 9.
3. Perform a range update: update range (1, 3) by adding 8 using lazy propagation.
4. Draw the final segment tree after all updates.

Solution

Step 1: Initial Array

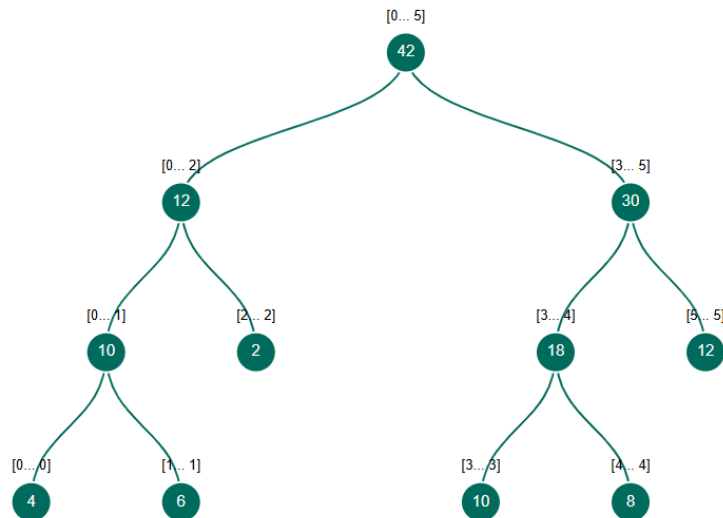
Index: 0 1 2 3 4 5

Array: [4, 6, 2, 10, 8, 12]

Step 2: Construct Segment Tree (RSQ)

Each node stores the **sum of its segment**.

Segment Tree Construction



Step 3: Point Update

Update index 2 with value 9

Old value at index 2 = 2

New value = 9

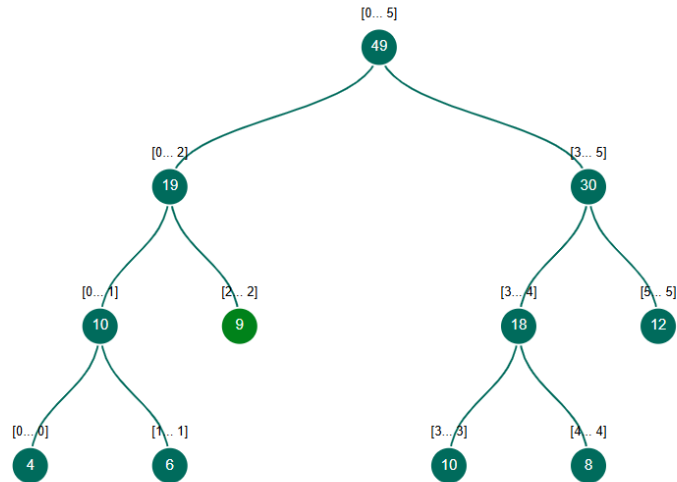
Updated array:

[4, 6, 9, 10, 8, 12]

Updated Segment Tree Nodes

- $[2-2] = 9$
- $[0-2] = 4 + 6 + 9 = 19$
- $[0-5] = 19 + 30 = 49$

Array elements	4	6	9	10	8	12
index	0	1	2	3	4	5



Step 4: Range Update (1, 3) += 8

Update indices **1** to **3** by adding **8**:

Index 1 $\rightarrow 6 + 8 = 14$

Index 2 $\rightarrow 9 + 8 = 17$

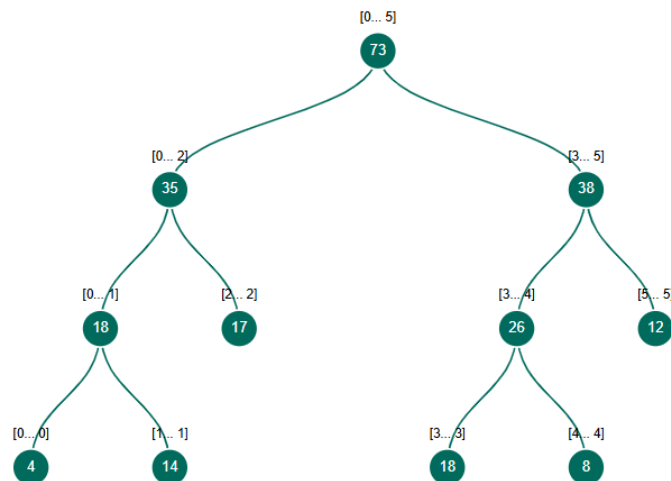
Index 3 $\rightarrow 10 + 8 = 18$

Final array:

[4, 14, 17, 18, 8, 12]

Array elements	4	6	9	10	8	12
index	0	1	2	3	4	5
range(1,3)=8		6+8	9+8	10+8		
New Array elements	4	14	17	18	8	12

Step 5: Final Segment Tree After All Updates



Final Updation:

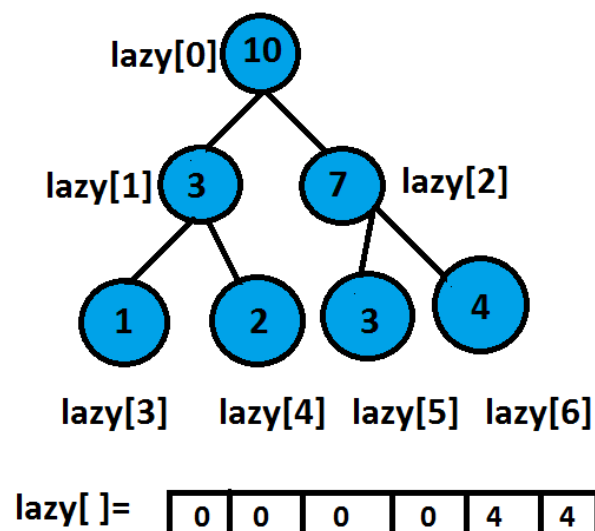
Operation	Result
Initial RSQ Tree Root	42
After Point Update	49
After Range Update	73
Final Array	[4, 14, 17, 18, 8, 12]

Lazy Propagation in segment tree

When there are many updates which are to be done on a range, we can postpone some updates (avoid recursive calls in updates) and do them only when required. We do this by creating an array named **lazy[]**, which stores the update information for the tree.

Example

Consider the node with value **7** in below diagram, this node stores sum of values at indexes from **2 to 3**. If our update query is for range **2 to 3** and we have to increase them by **4**, then we need to update this node and all descendants of this node. Using **Lazy Propagation**, we update only node with value **7** to **7+4=11** and store the updates to be done to its children in separate nodes called lazy nodes. The **lazy[]** array represents lazy node. Size of **lazy[]** is same as array that represents segment tree.



Firstly, initialize all elements of **lazy[]** as 0. It indicates that there are no pending updates on the **i**th node in segment tree. A non-zero value means that this amount needs to be added to the **i**th node in segment tree before making any query to the node.

To update an interval we make three cases:

1) If the current node has any pending updates then firstly add those updates to the lazy node. **2)** If the interval represented by current node lies completely in the interval to update, then update the current node and update the **lazy[]** array for children nodes. **3)** If the interval represented by current node overlaps with the interval to update, then- **a)** Recur for left and right children. **b)** Update the nodes as the earlier update function. We also need to update the **query()** function for any pending updates in the given query.